Student Performance Analytics System

Project Documentation | Python with AI | EWB Courses

1. Project Title

Project	Student Performance Analytics System
Student	Anuj Trivedi
Course	Python with AI - EWB Courses
Institution	Allenhouse Institute of Technology, Kanpur (B.Tech Computer Science, 2023-2027)
Technology	Python, NumPy, Pandas
Submission Deadline	15 October 2026

2. Objective

Schools and colleges collect large amounts of marks data, but raw numbers make it hard to see who is performing well, who needs support and which subjects are difficult. This project solves that problem with a Python program that reads student records from a CSV file, analyses them using NumPy and Pandas, and produces clear, meaningful results.

- Calculates total and average marks for every student
- Assigns grades and pass/fail status using clearly defined rules
- Analyses performance subject-wise and for the whole class
- Identifies the top-performing students
- Demonstrates course concepts: variables, data types, loops, functions, NumPy and Pandas

3. Technologies Used

Tool	Purpose
Python 3	Main programming language
NumPy	Numerical calculations (sum, mean, max, min, conditional grading)
Pandas	Loading the CSV, DataFrame handling, filtering, sorting and analysis
Git & GitHub	Version control and project submission

4. Dataset Description

File: students.csv **Records:** 20 students

Column	Description
Student_ID	Unique ID of each student (e.g. S001)
Name	Student name
Department	Department of the student
Python, SQL, HTML, CSS, JavaScript	Marks out of 100 in each of the five subjects
Attendance	Attendance percentage

The dataset contains 20 student records and is loaded into a Pandas DataFrame by main.py.

5. Implementation

Step 1 - Prepare the dataset

A students.csv file with 20 student records was created, containing student details, five subject marks and attendance.

Step 2 - Read the data

The CSV file is loaded with `pd.read_csv('students.csv')` and the complete dataset is displayed in the terminal using `df.to_string()` under the heading STUDENT DATA.

Step 3 - Process the data

Reusable functions perform the calculations. Loops and Pandas operations apply them to every student.

- A **for loop** over `df.iterrows()` picks the five subject marks of each student
- **Total marks:** `calculate_total()` adds the marks using a loop
- **Average marks:** `calculate_average()` uses `np.mean()`
- **Grade:** `assign_grade()` applies the grading rule below
- **Pass/Fail:** `check_pass_fail()` applies the pass condition below
- Total, Average, Grade and Result are added to the DataFrame as new columns

Grading rule (based on average marks)

Average Marks	Grade
90 and above	A+
80 to 89.99	A
60 to 79.99	B
40 to 59.99	C
Below 40	F

Pass/Fail condition: a student **passes** if their **average marks are 40 or more**; otherwise the student **fails**.

Step 4 - Perform analysis

The program answers practical questions from the dataset:

- What is the overall class average?
- Which student has the highest average, and which has the lowest?
- Which subject has the best class average? (`analyze_subject_performance()`)
- Who are the top 3 performers?

Step 5 - Final output

The program prints three clearly labelled sections - STUDENT DATA, STUDENT PERFORMANCE and FINAL OUTCOME - so the output can be understood without reading the code.

6. Key Features

- Loads and organises student data with a Pandas DataFrame
- Calculates total and average marks per student
- Assigns grades using a clearly defined grading rule
- Identifies pass/fail students using a clear condition
- Finds the class average, highest average and lowest average (with student names)
- Finds the best-performing subject from subject-wise averages
- Lists the top 3 performing students
- Uses reusable functions (functions.py), loops and NumPy operations

Function (functions.py)	What it does
calculate_total(marks_list)	Returns the sum of a student's marks using a loop
calculate_average(marks_list)	Returns the average of the marks using np.mean()
assign_grade(average)	Returns A+, A, B, C or F for a given average
check_pass_fail(average)	Returns Pass if average is 40 or more, else Fail
analyze_subject_performance(df, subjects)	Computes each subject's class average and returns the best subject

7. Output / Screenshots

Screenshots of the terminal output of main.py (python main.py) are shown below.

Output 1 - Student data and start of the performance table (python main.py)

```
C:\Users\anujt\Desktop\Student-Performance-Analytics-System> python main.py
Objective -> Dataset -> Pandas -> NumPy -> Functions -> Loops/Conditions -> Analysis -> Output -> Final Outcome
================================================
 STUDENT PERFORMANCE ANALYTICS SYSTEM
================================================

========= STUDENT DATA =========

  Student_ID        Name  Python  SQL  HTML  CSS  JavaScript
0    ST001   Anuj Trivedi      73   86    63   49          77
1    ST002   Rahul Sharma      42   95    55   73          92
2    ST003    Neha Verma      53   57    45   45          58
3    ST004   Priya Singh      87   70    74   58          37
4    ST005    Aman Kumar      56   87    36   58          78
5    ST006   Sneha Gupta      64   72    36   98          94
6    ST007   Vikash Yadav      55   67    46   92          56
7    ST008    Riya Patel      95   78    59   83          61
8    ST009    Arjun Mehta      93   97    76   98          62
9    ST010    Kavya Jain      94   50    49   96          96
10   ST011    Mohit Singh      81   96    85   78          89
11   ST012   Pooja Mishra      86   98    91   37          71
12   ST013    Rohit Kumar      85   41    55   43          73
13   ST014    Simran Kaur      52   38    59   94          48
14   ST015    Deepak Rao      84   92    43   60          87
15   ST016  Anjali Sharma      36   54    62   81          94
16   ST017    Karan Singh      41   78    95   42          81
17   ST018    Ishita Roy      69   48    51   70          84
18   ST019  Siddharth Nair      74   38    36   96          40
19   ST020    Tanya Desai      88   76    38   88          63

========= STUDENT PERFORMANCE =========

  Student_ID        Name  Total  Average Grade Result
0    ST001   Anuj Trivedi    348     69.6     B   Pass
1    ST002   Rahul Sharma    357     71.4     B   Pass
```

Output 2 - Performance table, final outcome and top 3 performers

```
2    ST003    Neha Verma    258     51.6     C   Pass
3    ST004   Priya Singh    326     65.2     B   Pass
4    ST005    Aman Kumar    315     63.0     B   Pass
5    ST006   Sneha Gupta    364     72.8     B   Pass
6    ST007   Vikash Yadav    316     63.2     B   Pass
7    ST008    Riya Patel    376     75.2     B   Pass
8    ST009    Arjun Mehta    426     85.2     A   Pass
9    ST010    Kavya Jain    385     77.0     B   Pass
10   ST011    Mohit Singh    429     85.8     A   Pass
11   ST012   Pooja Mishra    383     76.6     B   Pass
12   ST013    Rohit Kumar    297     59.4     C   Pass
13   ST014    Simran Kaur    291     58.2     C   Pass
14   ST015    Deepak Rao    366     73.2     B   Pass
15   ST016  Anjali Sharma    327     65.4     B   Pass
16   ST017    Karan Singh    337     67.4     B   Pass
17   ST018    Ishita Roy    322     64.4     B   Pass
18   ST019  Siddharth Nair    284     56.8     C   Pass
19   ST020    Tanya Desai    353     70.6     B   Pass

========= FINAL OUTCOME =========
Class Average: 68.60
Highest Average: 85.8 (Mohit Singh)
Lowest Average: 51.6 (Neha Verma)
Best Performing Subject: JavaScript

Top 3 Performers:
     Name  Average Grade
Mohit Singh    85.8     A
Arjun Mehta    85.2     A
 Kavya Jain    77.0     B
```

The final outcome shows a class average of 68.60, highest average 85.8 (Mohit Singh), lowest average 51.6 (Neha Verma), JavaScript as the best-performing subject, and Mohit Singh, Arjun

Mehta and Kavya Jain as the top 3 performers. All 20 students passed.

8. Final Outcome

The project successfully reads a student dataset and produces a complete performance report: total and average marks, grades, pass/fail status, highest, lowest and class-average performance, subject-wise analysis and a list of top performers. It shows practical use of Python functions, loops, NumPy and Pandas on a real-world style problem, and the output is readable without opening the source code.

9. Challenges & Learning

Challenge	How it was solved
Deciding clear grading and pass/fail rules	Defined a fixed grade table and a pass condition (average of 40 or more) and wrote them as separate functions
Applying the functions to every student	Used a loop over df.iterrows() to calculate total, average, grade and result for each row, then stored them as new DataFrame columns
Choosing between NumPy and Pandas	Used NumPy for calculations (mean, max, min) and Pandas for loading, sorting and displaying data
Making output readable	Printed labelled sections with headings and used to_string() for clean tables

What I learned: how to structure a project into functions, how NumPy and Pandas work together, how to analyse data to answer real questions, and how to publish and document a project on GitHub.

10. GitHub Repository

Repository link: https://github.com/anujtrivedi7589/Student-Performance-Analytics-System

Project structure

```
Student-Performance-Analytics-System/
|-- main.py          # Main file - runs the analytics
|-- functions.py     # Calculation logic
|-- students.csv     # Student dataset (20 records)
|-- README.md        # Project documentation
|-- Project_Documentation.pdf
```

How to run

```
pip install numpy pandas
python main.py
```